

Microservice Architecture

Шаблонизация манифестов. Helm. ArgoCD

otus.ru



Проверить, идет ли запись

Меня хорошо видно & слышно?



Ставим “+”, если все хорошо
“-”, если есть проблемы

Тема вебинара

Шаблонизация манифестов. Helm. ArgoCD.



Лавров Денис

Solution architect @ MTS AI

- 17 лет в ИТ
- занимался архитектурой инструментов модельного инференса в Сбере
- распиливал монолиты на микросервисы
- строил аналитические хранилища и средства визуализации отчетности
- преподаватель курсов «Инфраструктурная платформа на основе Kubernetes», «Микросервисная архитектура» и «SoftwareArchitect» в ОТУС

@tg roflmaoinmysoul

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#канал группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара

- Основные понятия
- Установка публичных чартов
- Создание своего чарта
- Helm в деталях
- GitOps и ArgoCD

Цели вебинара

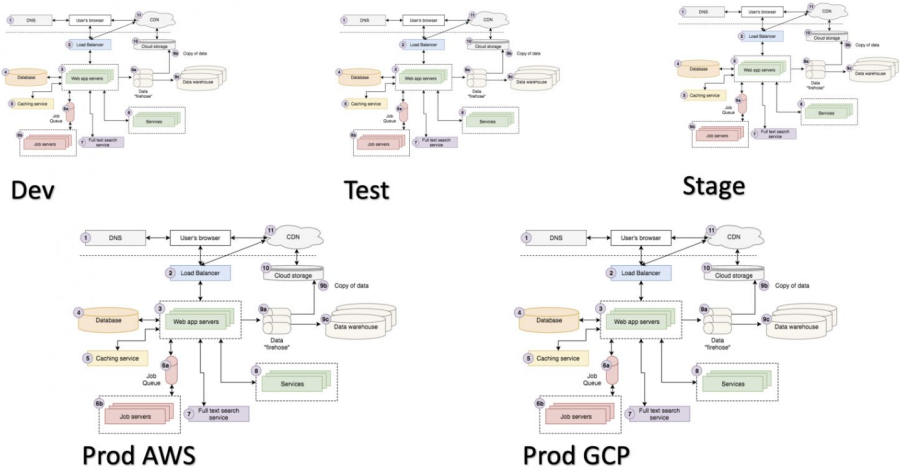
К концу занятия вы сможете

1. Установить публичный Helm chart
2. Создать свой собственный chart и задеплоить его в кластер
3. Конфигурировать устанавливаемые чарты

Основные понятия

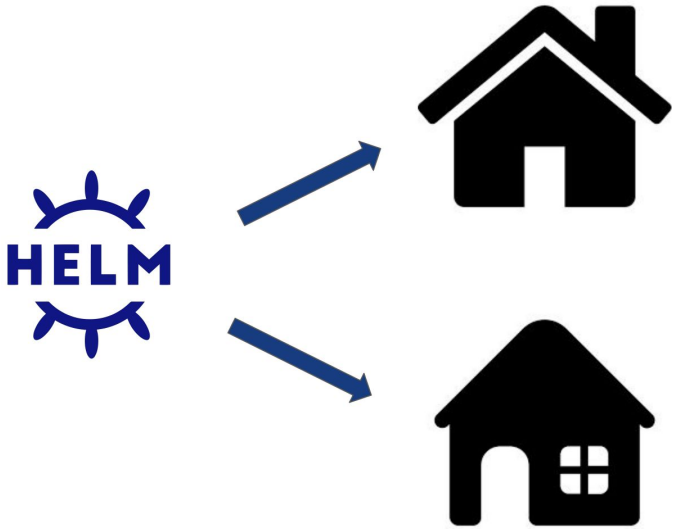
Проблемы уровня приложения в k8s

- Приложение состоит из большого количества kubernetes-сущностей
- Необходимость установки приложения в разные окружения или с различными настройками
- Возможно приложению для работы требуются какие-то зависимости
- Управление жизненным циклом приложения (обновление, откат и т.д)

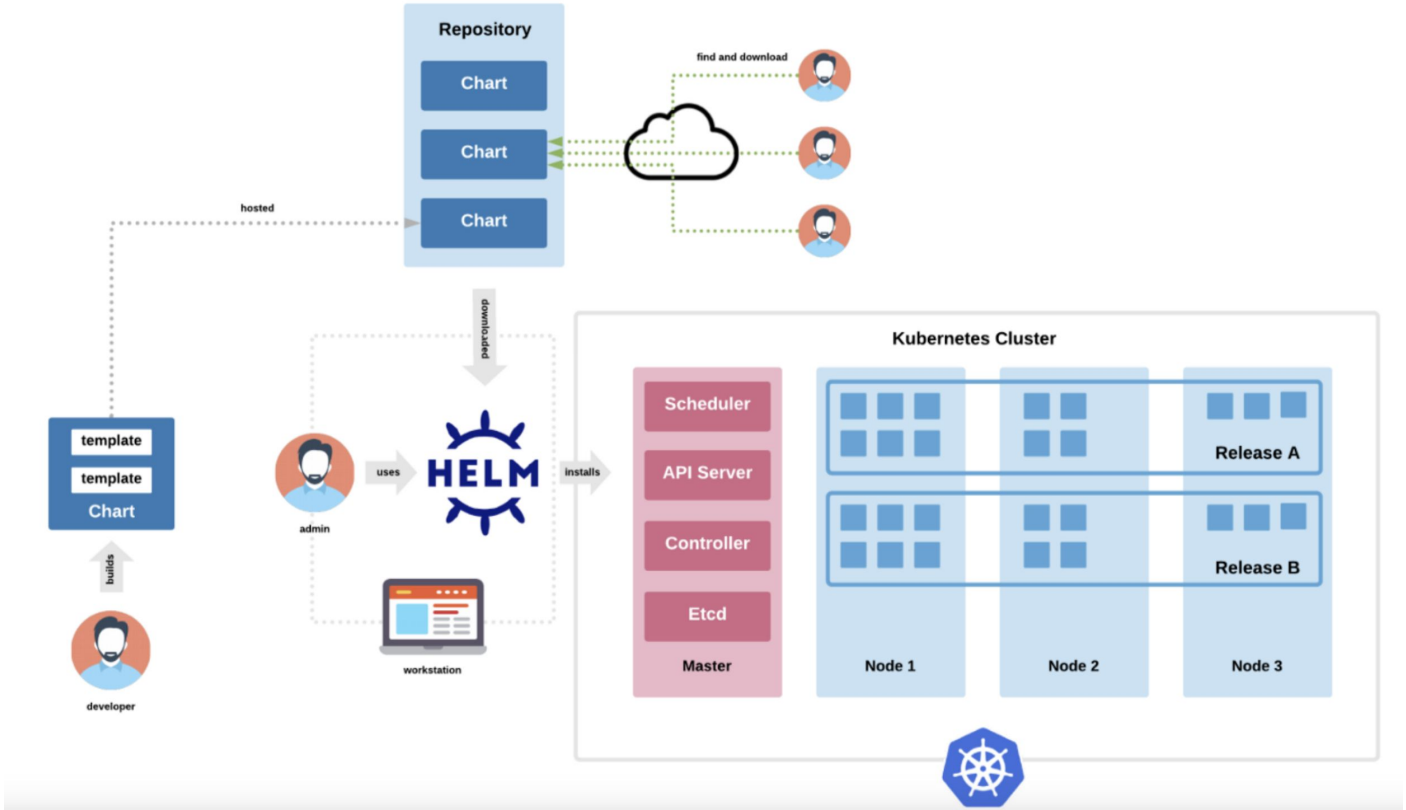


Что такое Helm

- Пакетный менеджер, позволяющий устанавливать (и не только) приложения, состоящие из большого количества Kubernetes-примитивов как единое целое
- В терминологии helm приложение называется чартом (chart)
- Чарты имеют фиксированную структуру и поддерживают простую текстовую шаблонизацию (go templates + sprig)
- Helm – не устанавливается в кластер, а является просто бинарником на машине разработчика/администратора



Архитектура Helm



Работа с публичными чартами

Как установить публичный чарт

Последовательность действий:

- Установить репозиторий

```
helm repo add [NAME] [URL] [flags]
```

- Установить чарт

```
helm install [NAME] [CHART] [flags]
```

- Можно задавать параметры

```
$ helm upgrade -f myvalues.yaml -f override.yaml redis ./redis
```

- Можно посмотреть историю

```
$ helm history angry-bird
REVISION    UPDATED                     STATUS          CHART
1           Mon Oct 3 10:15:13 2016     superseded      alpine-0.1.0
2           Mon Oct 3 10:15:13 2016     superseded      alpine-0.1.0
```

- И откатиться

```
helm rollback <RELEASE> [REVISION] [flags]
```

Задание параметров

- --set

```
$ helm install --set name=prod myredis ./redis
```

  - Определяет значение параметра
  - Можно задавать несколько подряд
  - Есть аналогичные методы –set-file, --set-json и т.д
- -f, --values

```
$ helm install -f myvalues.yaml -f override.yaml  myredis ./redis
```

  - Определяет yaml файл с значениями параметров
  - Можно задавать несколько
  - Если параметры в файлах конфликтуют – приоритет имеет тот, которые был написан позже

Задание параметров

- --set

```
$ helm install --set name=prod myredis ./redis
```

  - Определяет значение параметра
  - Можно задавать несколько подряд
  - Есть аналогичные методы –set-file, --set-json и т.д
- -f, --values

```
$ helm install -f myvalues.yaml -f override.yaml  myredis ./redis
```

  - Определяет yaml файл с значениями параметров
  - Можно задавать несколько
  - Если параметры в файлах конфликтуют – приоритет имеет тот, которые был написан позже

Протестировать перед применением

- template

```
helm template [NAME] [CHART] [flags]
```

  - генерирует манифесты на основе чарта и переменных локально без применения в кластер
- --dry-run

```
helm install --dry-run
```

  - генерирует манифесты на основе чарта и переменных локально без применения в кластер
  - Возможно задать обращение к кластеру на предмет конфликтов и поиска установленных уже версий приложения

Ревизии релиза и откат

- history

```
$ helm history angry-bird
REVISION    UPDATED                     STATUS          CHART
1           Mon Oct 3 10:15:13 2016     superseded      alpine-0.1.0
2           Mon Oct 3 10:15:13 2016     superseded      alpine-0.1.0
```

  - Позволяет посмотреть историю ревизий данного релиза
- Rollback

```
helm rollback <RELEASE> [REVISION] [flags]
```

  - Позволяет выполнить откат на конкретную ревизию релиза

Хранение параметров релиза в Helm

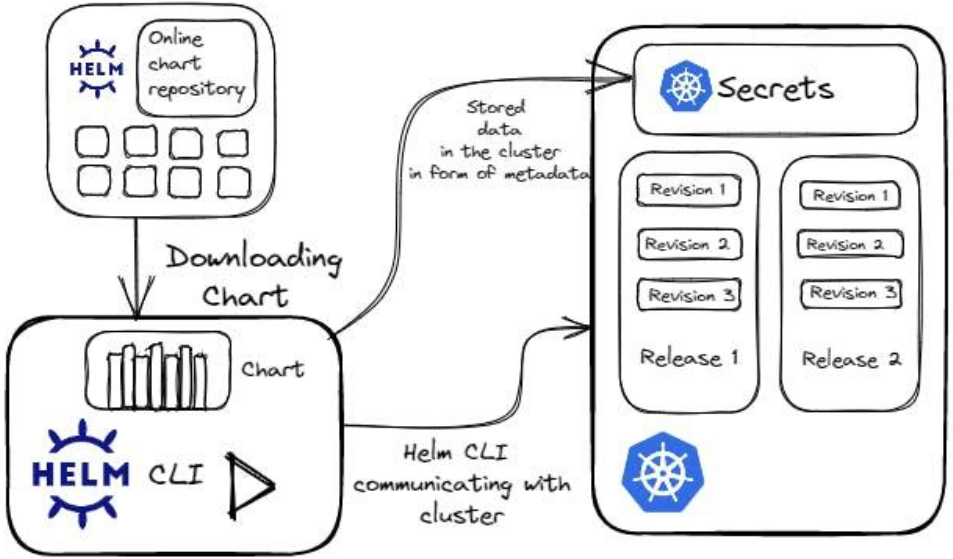
Helm 3 поддерживает 3 способа хранения информации о релизах:

- Secret – по умолчанию, хранится в виде JSON в gzip + Base64
- SQL
- Configmap – использовался в Helm 2

3 way merge strategy

Последовательность применения чарта в Helm 3

- Получить на вход чарт (локально или из удаленного репозитория)
- Получить предыдущий примененный чарт
- Получить текущее состояние объектов из кластера
- Сравнить эти 3 сущности, сгенерировать patch и передать его в kube API
- Дождаться выката (опционально)



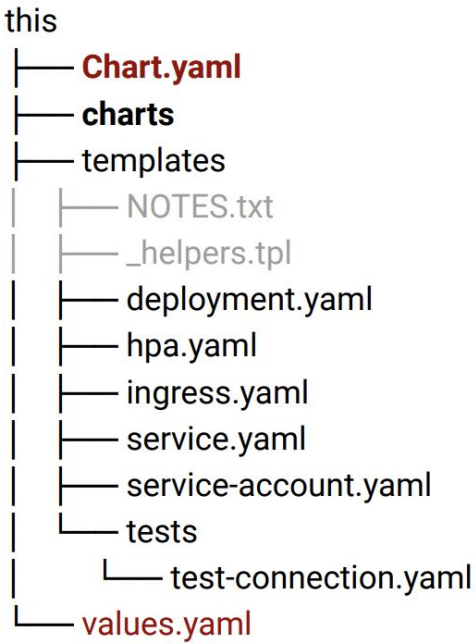
Создание своего чарта

Структура чарта

Helm create mychart

```
./mychart
├── Chart.yaml
├── charts
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml
```

Структура чарта: Chart.yaml




```
apiVersion: v2
name: this
description: A Helm chart for Kubernetes

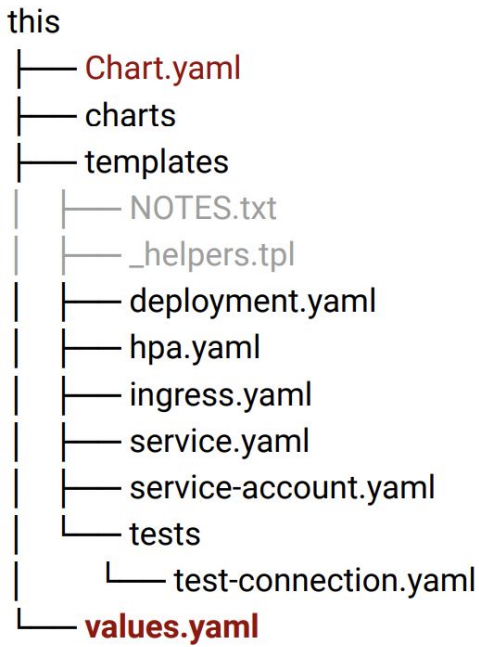
type: application


version: 0.1.0


appVersion: "1.16.0"
```

```
dependencies:
- name: redis
  repository: "bitnami/redis"
```

Структура чарта: Values.yaml




```
# Default values for this.
# This is a YAML-formatted file.
# Declare variables to be passed into your templates.

replicaCount: 1

image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: ""


...
```

Структура чарта: templates

$ helm create this

```
this
├── Chart.yaml
├── charts
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── service-account.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "this.fullname" . }}
  labels:
        {{- include "this.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  ports:
      - port: {{ .Values.service.port }}
        targetPort: http
        protocol: TCP
        name: http
  selector:
        {{- include "this.selectorLabels" . | nindent 4 }}
```

Структура чарта: NOTES

$ helm create this

```
this
├── Chart.yaml
├── charts
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── service-account.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml
```

```
1. Get the application URL by running these commands:
{{- if .Values.ingress.enabled }}
{{- range $host := .Values.ingress.hosts }}
  {{- range .paths }}
  http{{ if $.Values.ingress.tls }}s{{ end }}://{{ $host.host }}{{ .path }}
  {{- end }}
{{- end }}
```

```
$ helm install this this --set ingress.enabled=true
1. Get the application URL by running these commands:
  http://chart-example.local/
```

Структура чарта: _helpers.tpl

$ helm create this

```
this
├── Chart.yaml
├── charts
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── service-account.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml
```

```
{{/*
Expand the name of the chart.
*/}}
{{- define "this.name" -}}
{{- default .Chart.Name .Values.nameOverride |
trunc 63 | trimSuffix "-" }}
{{- end }}
```

Helm в деталях

Dependencies и sub-charts

- Независимое приложение, которое может устанавливаться отдельно от основного, или является его зависимостью
- В родительском чарте можно переопределять values в сабчартах
- Сабчарты не имеют доступа к values родительского
- Существуют глобальные переменные, доступные как из родительского так из сабчарта
- Объявляются в Chart.yaml либо в requirements

```
dependencies:
- name: nginx
  version: "1.2.3"
  repository: "https://example.com/charts"
- name: memcached
  version: "3.2.1"
  repository: "https://another.example.com/charts"
```

Хуки

- Позволяют управлять запуском определенных сущностей в конкретный момент жизненного цикла приложения
- Реализовано через аннотации вида helm.sh/hook
- Можно задавать веса, чтобы определять порядок запуска разных хуков (по-умолчанию 0)
- Если хук не выполнится успешно – релиз зафейлится (разное поведение для разных типов ресурсов)
- Ресурсы, создаваемые хуками не являются частью релиза
- Отдельные аннотации, определяющие политику удаления хука

Хуки. Доступные типы

- Pre-install
- Pre-delete
- Pre-upgrade
- Pre-rollback
- Test
- Post-install
- Post-delete
- Post-upgrade
- Post-rollback
-

Post-render

- Позволяет изменять созданные helm-ом манифесты перед тем, как они будут применены в кластер
- Может являться любым исполняемым файлом, которые принимает на вход манифесты, сгенерированные helm и возвращает валидные модифицированные манифесты
- Можно применять для команд install, upgrade и template

```
$ helm install mychart stable/wordpress --post-renderer ./path/to/executable
```

Минусы Helm

- Тяжело менять/расширять готовые чарты. Либо форкать либо post-render
- Строковая шаблонизация, отсутствует типизация переменных
- Низкий уровень абстракции зависимые от сущностей Kubernetes
- Не всегда предсказуемое поведение, большие чарты как black box

Альтернативы Helm

- **Kustomize** – встроенный в kubectl инструмент, позволяющий настраивать манифесты под различные задачи, не изменяя оригинальный yaml
- **Operators (CRD)** – механизм расширения функциональности Kubernetes за счет кастомных ресурсов и их контроллеров
- **Jsonnet** – based инструменты (eg ksonnet, tanka)
- **ArgoCD**, **flux**, **werf, helmfile**





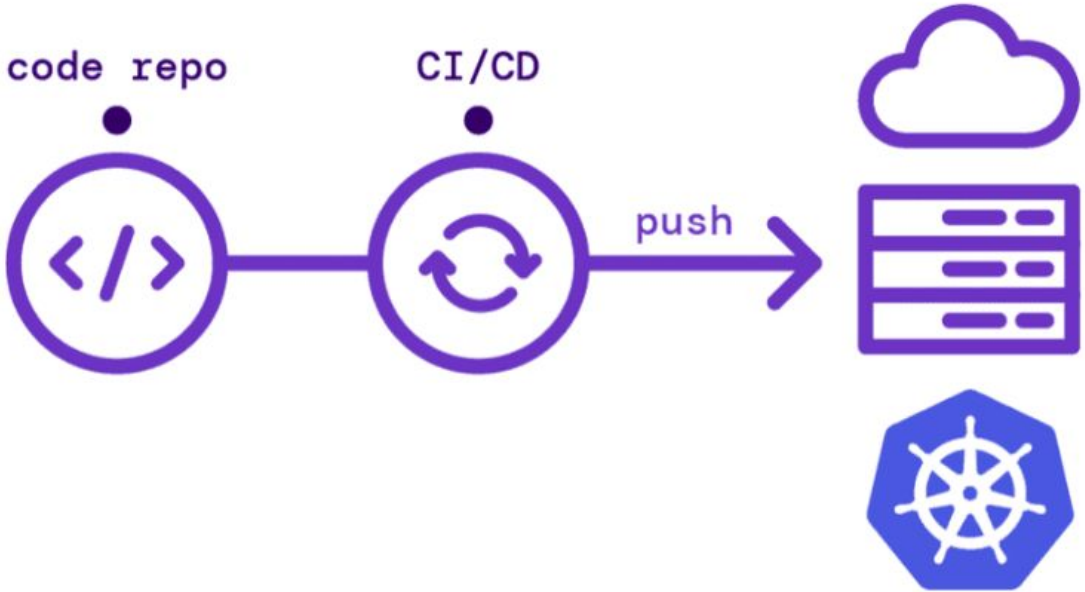


GitOps и ArgoCD

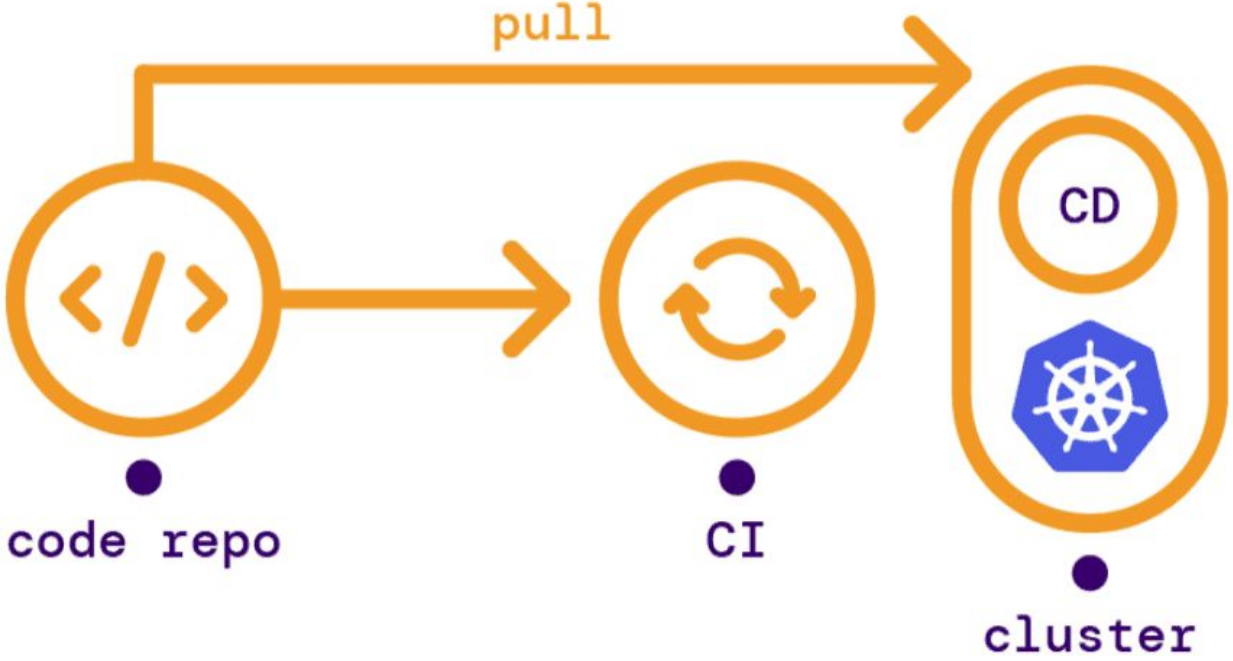
GitOps

- Концепция декларирующая единственным источником правды о состоянии кластера git-репозиторий
- Постоянный процесс синхронизации состояния кластера с желаемым состоянием, описанным в репозитории
- Доставка приложения до конечной среды исполнения более не является частью CD пайплайнов
- История git по сути становится аудит-логом изменения состояния кластера
- Можно деплоить как чистые манифесты, так и helm-чарты

Push based подход



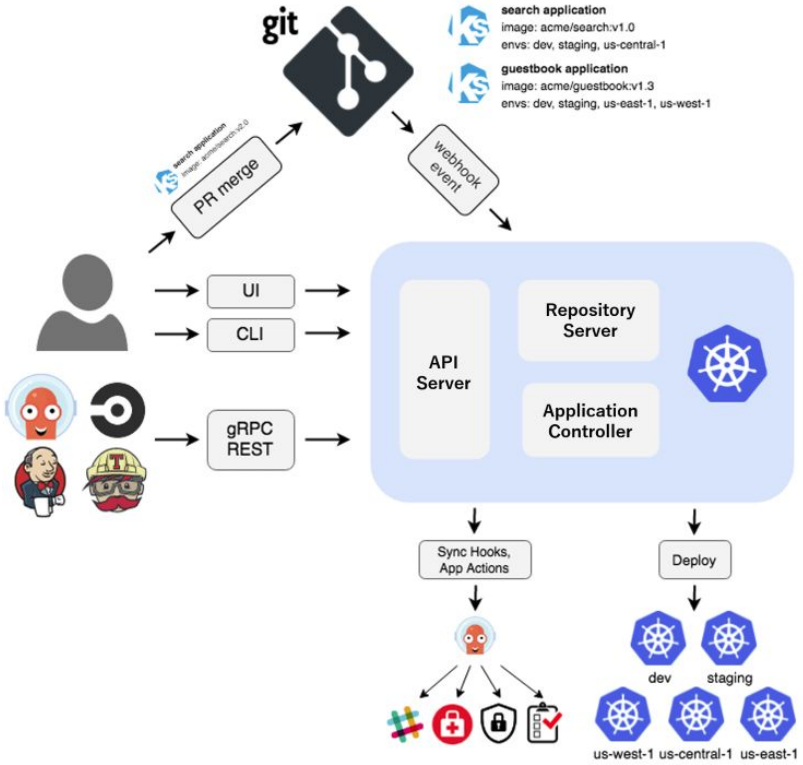
Pull based подход



AgroCD

- Инструмент, реализующий GitOps подход
- Устанавливается в кластер, может управлять раскаткой приложений в несколько кластеров, не только в тот, в котором установлен
- Отслеживает состояние в репозитории и автоматически или в ручном режиме синхронизирует реальное состояние кластера с желаемым
- Поддерживает Helm приложения из коробки
- Имеет достаточно продвинутый UI

AgroCD


git
search application
image: acme/search:v1.0
envs: dev, staging, us-central-1
guestbook application
image: acme/guestbook:v1.3
envs: dev, staging, us-east-1, us-west-1
PR merge
webhook event
UI
CLI
gRPC
REST
API Server
Repository Server
Application Controller
Sync Hooks, App Actions
Deploy
dev
staging
us-west-1
us-central-1
us-east-1

LIVE

Рекомендуемая литература

- Learning Helm: Managing Apps on Kubernetes
- Официальная документация Helm

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “–”,
если вопросов нет

Заполните, пожалуйста, опрос о занятии по ссылке в чате